Type Expansion in SKJS: the Instance Layer, the Expansion Cache, and Axiom **L-EXP**

Deep Dive C of the SKJS Soundness Series

Models revision `bafdcf7c` of `main` (2026-08-05).
Companion to the main soundness paper.

Abstract

The main soundness paper assumes axiom **L-EXP**: expansion of a type to its instance-plus-clauses view preserves or enlarges its semantics, with residual instances keeping theirs. We model expansion as a worklist over the instance layer and prove $[\![t]\!]_\rho \subseteq [\![\mathsf{expand}(t)]\!]_{\rho'}$ for the `None` and covariant modes used by refinement. Contravariant expansion does not satisfy this property, so it is excluded from L-EXP.

We also state and check the condition required to reuse entries in `seenExpansion`. Alias normalization does not consult existing variable bindings, so an alias expansion depends only on the resolved definition and its arguments. Cache reuse among references to the same definition is therefore exact. The implemented key, however, stores an unqualified name rather than definition identity. Same-named aliases in two modules can collide, producing an unsound acceptance in one expansion order and a false rejection in the other (**E-EXPKEY**). Class expansion does read existing bindings and remains a separate open obligation (**E-CLASS-SYM**). Finally, we examine recursion-guarded cache entries, the opaque type used for unannotated ambient fields (**E-PLACEHOLDER-ATOM**), and normalization of template literal types, `Closed<T>`, and enums.

Contents

1 Purpose **2**

2 Minimal Recap **2**
- 2.1 Representation and semantics . . . 2
- 2.2 The substitution state, condensed . . . 3
- 2.3 The axiom under study . . . 3

3 Expansion as a Function **4**
- 3.1 The worklist . . . 4
- 3.2 Containment through the worklist . . . 5

4 Per-Construct Containment **6**
- 4.1 Type variables . . . 6
- 4.2 Named instantiations (`TSInst`) . . . 7
- 4.3 Unexpanded intersections (`TInter`) . . . 7
- 4.4 Delayed array literals (`TArrayLit`) . . . 8
- 4.5 The containment theorem . . . 8

5 Centerpiece: Soundness of the `seenExpansion` Cache **8**
5.1 The key does not identify a definition (E-EXPKEY) 9
5.2 Symbolic arguments, precisely 9
5.3 The alias path is binding-blind 10
5.4 Verifying the comment's three claims 12
5.5 Where the proof stops: the class path 12
5.6 The missing-annotation placeholder atom 13

6 Alias Cycles and Termination **14**

7 Status and Ledger Edits **15**
7.1 Decomposition of L-EXP 15
7.2 Registry 16
7.3 Findings from failed proof attempts (index) 17

1 Purpose

Row addressed. **L-EXP** (the main paper's Assumptions and Proof Obligations table records the mode-scoped PROVED status this paper establishes): "Expansion preserves or enlarges semantics; residual instances keep theirs," anchored at `TType::expand`. Every refinement operator of the main paper runs on the expanded view (its Definition "Memberwise Refinement"), and the proof of its memberwise-refinement lemma consumes **L-EXP** in its first step; the axiom is therefore necessary for every branch-soundness result.

What this paper does. (i) models expand as a function of a type, a substitution state, and a variance mode, exactly following `TType::expandToVectors`; (ii) proves the containment per instance construct, splitting off two named interface assumptions where the argument crosses into `TEnv` (normalization) and array-literal commitment; (iii) proves the `seenExpansion` cache-invariance theorem for the type-alias and enum paths, names the class path as an obligation, and refutes the cache *key*'s identity premise across modules (E-EXPKEY, witnessed); (iv) records the state evolutions expansion can cause and defers their transport to the main paper's **L-MONO**; (v) audits the missing-annotation placeholder atom (E-PLACEHOLDER-ATOM) and states precise ledger edits (Section 7). Failed proof attempts are reported as numbered Findings.

Conventions. Notation, semantics, and the status vocabulary (PROVED / ASSUMED / OPEN / VIOLATED) are the main paper's. All code references are against the pinned revision `bafdcf7c`; line numbers are cited where they disambiguate. "Paper B" is the deep dive on union and intersection (**L-UNION**/**L-INTER**/**L-INTER-BOT**), which this paper cites and does not re-prove.

2 Minimal Recap

2.1 Representation and semantics

A TType is a pair (I, C) of an instance set $I \in \mathsf{TInsts}$ and clauses $C \in \mathsf{Clauses}$, with $[\![(I, C)]\!]_\rho = [\![I]\!]_\rho \cup [\![C]\!]_\rho$ (main paper, Definition "Clause, Clauses, TType"). A clause $\langle V \mid S \rangle$ pairs type parameters with an STypes union or $\top$; the top clause $\langle \emptyset \mid \top \rangle$, if present, is alone. The instance layer TInst has four constructors: type variables α (`TVar`, carrying a finite set of *occurrence-level* constraint types — `TVar::setConstraint` is a functional update returning a new `TVar` value with

the same identifier — registry row SETCONSTRAINT-FUN, deep dive E), unexpanded named instantiations $\texttt{TSInst}(n, \bar{a})$, unexpanded intersections $\texttt{TInter}(\{t_1, \ldots, t_k\})$, and delayed array literals $\texttt{TArrayLit}(\alpha, \mathit{elts}, \mathit{fresh})$.

impl: `TType.sk / TType, TInsts, Clause, Clauses`

impl: `TType.sk / TVar (2576), TSInst (2626), TArrayLit (2618); TTypeInter.sk / TInter (17)`

The main paper gives the instance layer an abstract, state-indexed semantics. We spell out the part this paper consumes:

Definition 1 (Instance Semantics Interface)**.** *For a substitution state ρ (Definition 2):*

(V1) $[\![\alpha^K]\!]_\rho \subseteq \bigcap_{c \in K} [\![c]\!]_\rho$ *for a variable occurrence α^K with constraint set K;*

(V2) if ρ binds α (as $\mathsf{SVOpen}(t)$ *or* $\mathsf{SVClosed}(t)$*), then* $[\![\alpha^K]\!]_\rho \subseteq [\![t]\!]_\rho$*;*

(I1) $[\![\mathsf{TInter}(\{t_i\})]\!]_\rho = \bigcap_i [\![t_i]\!]_\rho$ *(an unexpanded intersection* means *the intersection);*

(N1) $[\![\mathsf{TSInst}(n, \bar{a})]\!]_\rho$ *is the denotation of the named definition n at arguments $\bar{a}$ — related to expansion only through assumption* ***E-NORM*** *below;*

(A1) $[\![\mathsf{TArrayLit}(\alpha, \mathit{elts}, _)]\!]_\rho$ *is the set of runtime values of the originating array-literal expression — related to expansion only through assumption* ***E-ARR*** *below.*

(V1)/(V2) are the main paper's canonical variable denotation, registry row TVAR-SEM: a bound occurrence denotes its envelope intersected with its occurrence constraints, and an unbound occurrence denotes $\emptyset$. This paper imports that definition.

2.2 The substitution state, condensed

Definition 2 (State; get transitions used by expansion)**.** *ρ maps* `TVarIDs` *to* $\mathsf{SVOpen}(t)$ *or* $\mathsf{SVClosed}(t)$ *and carries the flag* freeze *(*`freezeCandidates`*,* not *part of the saved state). Expansion reads variables through* $\mathsf{get}(\alpha, \mathit{var})$*: covariant get on an* unbound *α seals it to top ($\alpha \mapsto \mathsf{SVClosed}(\top)$, returning $\top$); on* $\mathsf{SVOpen}(t)$ *it seals to* $\mathsf{SVClosed}(t)$ *and returns t — except under* freeze *for a candidate, where it returns t* without *sealing; on* $\mathsf{SVClosed}(t)$ *it returns t. Contravariant get returns $\bot$ on unbound (no write) and the bound value otherwise. Both then intersect the result with the occurrence's constraint types via* `TType::inter`*. The full transition table is the main paper's Definition "Transitions".*

impl: `TTypeSubst.sk / Subst::get (188), freezeCandidates (89)`

The mutable `Subst` also carries four memo caches — `seenSub`, `seenInter`, `seenUnion`, and this paper's subject `seenExpansion` : $\mathsf{ExpansionKey} \to \mathsf{TType}$, where $\mathsf{ExpansionKey}(n, \bar{a})$ is the *name text* of a named type plus its argument list (Section 5.1) — all four part of the saved state: `save`/`restore` and `clone` copy bindings and caches *together*.

impl: `TTypeSubst.sk / SubstState (13--49), SavedSubst (51), clone (149)`

impl: `TType.sk / ExpansionKey (3055), expansionCacheKey (3057)`

2.3 The axiom under study

Axiom 3 (L-EXP, as stated in the main paper)**.** *Expansion rewrites a type to an instance-plus-clauses view using the current substitution state, possibly evolving it ($\rho \to \rho'$, e.g. sealing a fresh covariantly-read variable to top). Then* $[\![t]\!]_\rho \subseteq [\![\mathsf{expand}(t)]\!]_{\rho'}$*, and residual instances keep their semantics.*

Two things the main paper leaves open, which this paper fixes: expansion takes a *mode* argument (`expandTVars : ?Variance`), and the containment holds for modes none and cov but *not* for contra (Section 4.1); and "residual instances" means exactly: unresolved type variables in mode none, plus unexpanded recursion cut-offs re-emitted by `instantiate` and `TInter` cycle guards (Sections 4.2, 4.3, 6).

3 Expansion as a Function

3.1 The worklist

Definition 4 (Expansion)**.** *For a mode* $m \in \{\mathsf{none}, \mathsf{cov}, \mathsf{contra}\}$ *(the implementation's* ***expandTVars*** $\in \{$***None()***, ***Some(Covariant())***, ***Some(Contravariant())***$\}$*), define* $\mathsf{expand}_m(t, \rho) = (I', C', \rho')$ *by the following worklist, mirroring* ***TType::expandToVectors*** *line by line. If* t *is top, the result is* $\top$ *(returned as* ***None****). Otherwise initialize* $C' := t.\mathsf{clauses}$ *(as a vector),* $I' := \emptyset$*, todo* $:= t.\mathsf{insts}$*, seen* $:= t.\mathsf{insts}$*. While todo is nonempty, pop* i *and compute a one-step expansion* $E_m(i)$*:*

- $i = \mathsf{TSInst}(n, \bar{a})$*: let* $k = \mathsf{ExpansionKey}(n.\mathsf{text}, \bar{a})$*; if cached,* $E_m(i) := \rho.$***seenExpansion***(k)*; else* $E_m(i) := \mathsf{instantiate}(n, \bar{a})$ *and the cache is extended (*$\rho.$***seenExpansion***$[k] := E_m(i)$*). Mode-independent. The key is the definition's* spelling*, not its resolution — the gap between the two is Section 5.1.*
- $i = \mathsf{TInter}(\{t_j\})$*:* $E_m(i) := \mathsf{interN}(\{t_j\})$ *with* ***tenv.expandInter*** *set (Section 4.3). Mode-independent.*
- $i = \alpha^K$*: in mode* none*, push* α^K *onto* I' *(*residual*) and continue; in modes* cov/contra*,* $E_m(i) := \mathsf{get}(\alpha^K, m)$ *— the binding read intersected with the occurrence constraints, with the seal side effects of Definition 2.*
- $i = \mathsf{TArrayLit}(\alpha, \mathit{elts}, _)$*:* $E_m(i)$ *is the committed type* $\mathsf{peek}(\alpha)$ *if* α *is bound (no state change), else the mutable tuple type over elts (Section 4.4). Mode-independent.*

If $E_m(i)$ *is top, the whole expansion returns* $\top$*. Otherwise extend* C' *with* $E_m(i).\mathsf{clauses}$ *and push each instance of* $E_m(i).\mathsf{insts}$ *not in seen onto todo (adding it to seen). On termination,* $\mathsf{expand}_m(t, \rho) = (I', C', \rho')$ *where* ρ' *is the evolved state.* ***TType::expand*** *rebuilds the pair from* $\mathsf{TInsts::unionN}(I')$ *and* $\mathsf{Clauses::unionN}(C')$*;* ***expandToClauses*** *does the same in mode* ***Some(variance)*** *and returns the clause part. A second site performs the same keyed expand-and-cache step outside this worklist:* ***growTVars****'s* ***collectTVars*** *expands* ***TSInst*** *entries one level to find inference variables hidden inside head-transparent aliases; it shares the map and the key, so everything this paper proves (and refutes) about reuse applies to it verbatim.*

impl: `TType.sk / expandToVectors (680), expand (727), expandToClauses (739)`

impl: `TTypeSubtyping.sk / collectTVars (1508, cache use 1529--1539)`

Lemma 5 (State Evolutions of Expansion)**.** *The only substitution evolutions* expand_m *performs are: (i)* get *transitions — seal-to-top of unbound variables and value-preserving seals* $\mathsf{SVOpen}(t) \to \mathsf{SVClosed}(t)$ *(modes* cov*; mode* contra *performs no binding writes); (ii) memo-cache extensions (*`seenExpansion` *here;* `seenInter` */* `seenUnion` *inside the invoked operators); (iii) version bumps. In particular expansion never executes a* grow*: no binding value is ever unioned or replaced.*

Proof. By inspection of Definition 4 against the code: the `TVar` arm calls `Subst::get` only; the `TSInst` arm calls `tenv.instantiate(subst, subst, ...)`, and `instantiate` immediately repackages the state into a fresh `Subst::mcreate(tcenvb, baseSubst)` *copy* — its binding writes

(constraint-check clones, class-body expansion) land in the copy and are dropped; only the caller's `seenExpansion` write at `TType.sk:700` touches the live state. The `TInter` and `TArrayLit` arms call `interN` (get-transitions inside `inter_`'s covariant expansions, plus `seenInter`) and `peek` (read-only). □

impl: `TEnv.sk / instantiate (168, 196); TType.sk / expandToVectors (680); TTypeSubst.sk / peek (243)`

Transport of already-installed types across these evolutions is exactly the main paper's axiom **L-MONO** (whose anchor names the `Subst::get` seal-to-top); we cite it rather than re-prove it. Note that the value-preserving seal $\mathsf{SVOpen}(t) \to \mathsf{SVClosed}(t)$ does not change any denotation under Definition 1 — (V2) reads the bound value irrespective of open/closed — so among expansion's evolutions only seal-to-top has semantic content, and it only *adds* a binding of value $\top$.

3.2 Containment through the worklist

Lemma 6 (Worklist Containment)**.** *Fix a mode m and suppose the one-step containment*

$$[\![i]\!]_{\hat{\rho}} \subseteq [\![E_m(i)]\!]_{\hat{\rho}'} \qquad (*)$$

holds for every instance i popped during the run, where $\hat{\rho} \to \hat{\rho}'$ is that step's evolution. Then $[\![t]\!]_{\rho} \subseteq [\![(I', C')]\!]_{\rho'}$, and residual instances appear in I' verbatim.

Proof. Let $v \in [\![t]\!]_{\rho} = [\![t.\mathsf{insts}]\!]_{\rho} \cup [\![t.\mathsf{clauses}]\!]_{\rho}$. All later evolutions are those of Lemma 5, so membership transports forward by **L-MONO**; we may therefore evaluate all containments at the final ρ'. If v is justified by the clause part, it is in the initial C', and Clauses::unionN only merges clauses keyed by their var-sets using STypes/clause union, which over-approximates set union by **L-UNION** (paper B); the constructor invariant that a top clause absorbs the rest, and `TType::create`'s dropping of I' when C' is top, only enlarge the result. If v is justified by an instance i, argue by induction on the number of worklist steps after i enters *seen*: when i is popped, $(*)$ moves v into $[\![E_m(i)]\!]$, i.e. into its clauses (already covered) or one of its instances i''; if i'' is fresh it enters the worklist and induction applies; if $i'' \in$ *seen*, then i'' was either already processed (induction applies to its earlier occurrence — one-step expansion of the same instance under the evolved state yields the same $E_m(i'')$, by cache determinism for `TSInst` (Theorem 19) and by value determinism of sealed reads for the other arms) or is currently queued. The residual `TVar` arm pushes i itself onto I'; the top short-circuit returns $\top$ with $[\![\top]\!] = \mathsf{Val} \supseteq$ everything. □

One genuine subtlety hides in the "already processed" case: a *recursive* named type re-emits its own `TSInst`, which the *seen* check then drops — the recursive occurrence contributes nothing new only under a least-fixpoint reading of recursive definitions. Section 6 isolates this.

Lemma 7 (mapSTypes Decomposition)**.** *`TType::mapSTypes`(m, f) equals: expand in mode m; apply f to each clause's `?STypes` (with a $\top$ expansion presented to f as the single top clause `None`); drop clauses whose image is empty; rebuild with* Clauses::unionN *and the untouched residual I'. Consequently the main paper's refinement primitive* $\mathsf{refine}^{g,d}$ *factors exactly as its Definition "Memberwise Refinement" describes, with this paper supplying the expansion step and paper B the rebuild unions; and `mapSTypes` with the identity mapper is `expand`.*

impl: `TType.sk / mapSTypes (434--457); TypeCheckRefine.sk / setLocalFilterMap (721, 736)`

Proof. By inspection: `mapSTypes` calls `expandToVectors`, maps f through `Clause::create` (which returns `None` exactly on an empty STypes image, implementing the drop), and rebuilds. The top case at line 440–445 presents `None` to f and produces either the replacement clauses or $\top$. `rawMap`

is the substitution-free analogue used by generic transformers and performs no expansion at all (its `fInsts` may convert instances to clauses but has no access to a substitution); no refinement client reaches it, so it carries no **L-EXP** obligation.

impl: `TType.sk / rawMap (408)` □

4 Per-Construct Containment

4.1 Type variables

Lemma 8 (TVar One-Step Containment, modes none and cov)**.** *For every variable occurrence α^K and state ρ:*

(a) Mode none*: the step is the identity (α^K residual); containment and residual-semantics preservation are trivial.*

(b) Mode cov*, ρ binds α (open or closed, freeze or not): the step returns $t \sqcap K$ built by* ***TType::inter*** *from the bound value t and the occurrence constraints. By (V2), (V1) and* ***L-INTER/L-INTER-BOT*** *(paper B), $[\![\alpha^K]\!]_\rho \subseteq [\![t]\!]_\rho \cap \bigcap_K [\![c]\!]_\rho \subseteq [\![t \sqcap K]\!]_{\rho'}$, the last step reading the result at the post-step state via* ***L-MONO****. The seal $\mathsf{SVOpen}(t) \to \mathsf{SVClosed}(t)$, when it fires, is value-preserving.*

(c) Mode cov*, α unbound: the step seals $\alpha \mapsto \mathsf{SVClosed}(\top)$ and returns $\top \sqcap K$. By (V1) and* ***L-INTER/L-INTER-BOT****, $[\![\alpha^K]\!]_\rho \subseteq \bigcap_K [\![c]\!]_\rho \subseteq [\![\top \sqcap K]\!]_{\rho'}$; the new binding of value $\top$ makes (V2) hold vacuously at ρ'.*

Finding 9 (Contravariant expansion does not satisfy L-EXP)**.** *We first attempted to prove the one-step containment for all three modes. It fails in mode* contra *on an* unbound *variable:* $\mathsf{get}(\alpha, \mathsf{contra})$ *returns $\bot$ (no write), and $[\![\alpha^K]\!]_\rho \subseteq [\![\bot]\!] = \emptyset$ is false whenever the variable's envelope is inhabited. This is not a bug: a contravariant read is a deliberate* under*-approximation — "the lower bound gathered so far" — consumed by the subtyping/growth machinery (* ***growTVars****,* ***maybeGetRestElementType****), where the required direction is reversed; that is the main paper's Obligation "Validation Soundness", not this paper's. For a* bound *variable, case (b)'s argument goes through verbatim in mode* contra*, since* $\mathsf{get}(\mathsf{contra})$ *then returns the same bound value. Consequence: axiom* ***L-EXP*** *must be scoped to modes* none/cov *(the mode scoping of Axiom* ***L-EXP****; Section 7).*

impl: `TTypeSubst.sk / Subst::get (210--216); TTypeSubtyping.sk / growTVars`

Lemma 10 (Refinement Clients Use Only Modes none/cov)**.** *Every expansion performed by the refinement and constant-removal machinery of the main paper uses mode* cov *or mode* none*:*

- ***setLocalFilterMap*** *maps with* ***Some(Covariant())*** *(`TypeCheckRefine.sk:736`);*
- *the truthiness residues go through* ***residuePart*** *with* ***Some(Covariant())*** *(`TType.sk:391, 396`);*
- ***conditionVerdict*** *and* ***getSTypes****-based readers use* ***expandToClauses*** *with* ***Covariant()*** *(`TType.sk:319, 325, 676`);*
- ***removeConstant*** *expands with* ***None()*** *(`TType.sk:252, 258`), leaving variables residual and untouched, as its model in the main paper's Definition "Constant Removal" requires ("the instance layer is unchanged").*

So the mode scoping of Finding 9 costs the main paper nothing.

4.2 Named instantiations (`TSInst`)

$\mathsf{instantiate}(n, \bar{a})$ dispatches on the definition of n (`TEnv::instantiate`): a `TypeDef` binds each type parameter P_i to the argument a_i *as a type term* in the lexical `bindings` map (checking constraints on a throwaway clone, diagnostics only) and normalizes the alias body — a reference to P_i in the body resolves to the binding by `boundTy.withPos(pos)`, which changes only the position and not the hash, so the argument term is inserted *verbatim*; a `Class` builds the object type via `new`/`newObject`/`addFields` with the same binding mechanism for field types; an `Enum` builds the union of its member literal types (binding-blind — Lemma 18); an enum-*member* type reference (`E.A`) never reaches `instantiate` at all (it is resolved inside `normalize`'s `TAInst` arm, an `invariant_violation` guards the dispatch). Three early returns produce something other than the definition's body: the synthetic missing-annotation placeholder returns an opaque atom (Section 5.6); the recursion guard, when n is already being expanded (`seenInstantiate`), and the circular-alias guard (`isCircularTypeDef`, for an alias that requires itself) both re-emit the unexpanded TSInst (Section 6).

impl: `TEnv.sk / instantiate (168--231), instantiateTParams (257), normalize TAInst arm (2011--2116), new (842), newObject (901), isCircularTypeDef (495)`

impl: `TypeCheckEnum.sk / enumUnionType (487), enumMemberType (533)`

What normalization computes from the body is `TEnv` territory: this paper does not model `normalize` and instead names the interface assumption that the containment needs.

Axiom 11 (E-NORM: Instantiation is Semantics-Preserving)**.** *For every named definition n and arguments $\bar{a}$, and every state ρ at which the expansion runs, $[\![\mathsf{TSInst}(n, \bar{a})]\!]_\rho \subseteq [\![\mathsf{instantiate}(n, \bar{a})]\!]_\rho$. For the two identity early returns (recursion guard, circular alias) the result* is *the input, so the axiom is trivially true there and the re-emitted instance is a residual that keeps its semantics by construction; the placeholder-atom return is tracked separately (Section 5.6, registry row E-PLACEHOLDER-ATOM).*

impl: `TEnv.sk / instantiate, normalize`

Under **E-NORM**, the `TSInst` one-step containment reduces to one further claim: that reusing a *cached* expansion — computed at some earlier state ρ_0 — denotes at ρ' no less than a fresh `instantiate` run at ρ' would. That is the centerpiece (Section 5); Theorem 19 discharges it for the alias path by showing the cached and fresh values are *equal*.

4.3 Unexpanded intersections (`TInter`)

The one-step expansion of $\mathsf{TInter}(\{t_1, \ldots, t_k\})$ sets `tenv.expandInter` and left-folds `TType::inter` over the members. (That flag selects full over deferred field intersection; its path-scoping and the deferred form's containment are registry row TENV-EXPANDINTER (OPEN), which this arm's argument imports.) By (I1) and **L-INTER**/**L-INTER-BOT** (paper B, whose two cases together cover `inter` at every level, including the `seenInter` cache and the cycle guard `tenv.seenInter`),

$$[\![\mathsf{TInter}(\{t_i\})]\!]_\rho = \bigcap_i [\![t_i]\!]_\rho \subseteq [\![\mathsf{interN}(\{t_i\})]\!]_{\rho'},$$

with the fold's intermediate evolutions transported by **L-MONO**. When `inter_` hits an in-progress pair it returns the *unexpanded* intersection `unexpandedInter` (`TTypeInter.sk:154--156`); by (I1) that residual keeps its semantics exactly. Similarly the bare-variable arms return a variable occurrence with an added constraint — a residual whose semantics (V1)-intersects the operand, again exact for our direction.

impl: `TTypeInter.sk / TInter::expand (37), inter (97), inter_ (129), unexpandedInter (82)`

4.4 Delayed array literals (`TArrayLit`)

`ArrayLitTyping::expand` returns the committed type $\mathsf{peek}(\alpha)$ — the binding of the literal's commitment variable intersected with its constraints, read *without* any state transition — when α is bound, and otherwise the mutable tuple type $\mathsf{Tuple}[elts]$ built from the widened element types. The containment needs the commitment discipline documented at the top of `TArrayLitTyping.sk` ("every commitment uses the target type or the widened element types"), which is array-literal-typing territory; we name it:

Axiom 12 (E-ARR: Array-Literal Commitment Discipline)**.** *For every* $\mathsf{TArrayLit}(\alpha, elts, _)$ *and state* ρ*: (i) if* ρ *binds* α *then* $[\![\mathsf{TArrayLit}(\ldots)]\!]_\rho \subseteq [\![\mathsf{peek}(\alpha)]\!]_\rho$ *(the committed type contains the literal's values); (ii) if* α *is unbound then* $[\![\mathsf{TArrayLit}(\ldots)]\!]_\rho \subseteq [\![\mathsf{Tuple}[elts]]\!]_\rho$ *(the literal's element values inhabit the widened element types — a well-typedness fact about how* ***`TArrayLit`*** *is constructed). Clause (ii) must hold at every state, including states where* α *later becomes bound: the tuple view taken by an early expansion must not be invalidated by a later, wider commitment (e.g. to* ***`Array<U>`****).*

impl: `TArrayLitTyping.sk / expand (100), arrayLitCommittedType (85)`

impl: `TArrayLitTyping.sk / arrayLitToTupleType (113); TTypeSubst.sk / peek (243)`

4.5 The containment theorem

Theorem 13 (T-EXP)**.** *For* $m \in \{\mathsf{none}, \mathsf{cov}\}$*, all* t*,* ρ*: with* $(I', C', \rho') = \mathsf{expand}_m(t, \rho)$*,*

$$[\![t]\!]_\rho \subseteq [\![(I', C')]\!]_{\rho'},$$

residual instances appear verbatim in I' *and keep their semantics, and the state evolution* $\rho \to \rho'$ *consists only of the transitions of Lemma 5. The result is conditional on:* ***L-UNION/L-INTER/L-INTER-BOT*** *(paper B),* ***L-MONO*** *(main paper),* ***E-NORM***, ***E-ARR****, and — for reuse of cached* ***`TSInst`*** *expansions — the key-identity premise* ***E-EXPKEY*** *(Section 5.1;* violated*, see the registry), Theorem 19 on the alias path, and Obligation 25 on the class path.*

Proof. Lemma 6 reduces the claim to the one-step containment $(*)$, established per construct: Lemma 8 (variables), Axiom 11 plus Theorem 19 (named instantiations, cached or fresh), Section 4.3 (intersections), Axiom 12 (array literals). □

5 Centerpiece: Soundness of the `seenExpansion` Cache

The cache maps an ExpansionKey — the name *text* of a named type plus its argument list — to the expanded body, and is consulted before every instantiate call at both expansion sites (Definition 4). One entry therefore serves *every mention* of a named type under a substitution; for the zero-argument aliases that dominate real programs, the entry is filled once and hit everywhere. Exactness of that reuse rests on two independent claims: (i) the cached *value* does not depend on the state at fill time — the code comment (`TTypeSubst.sk:17--31`) argues this holds because the cached body substitutes each type parameter *symbolically*: arguments appear as free variables, re-resolved at every use through `Subst::get`; and (ii) the *key* identifies one definition, so the entry is only ever reused for the definition that filled it. This section makes both precise. For (i) it proves a strictly stronger statement on the alias path (binding-blindness) and corrects the comment in two places; for (ii) it exhibits a refutation — the key's name-text component conflates same-named definitions from different modules, witnessed end-to-end (Section 5.1).

5.1 The key does not identify a definition (E-EXPKEY)

ExpansionKey$(n, \bar{a})$ is built from `name.value` — the *unqualified* name text — and the argument list (`TType.sk:3055--3059`). Name *resolution* is per-module and correct: instantiate resolves through `defMap.getType` at the expansion site, and diagnostics anchor each reference at its own module's definition. The cache, however, is shared across the whole substitution, so two *different* definitions with one spelling and one argument list share one entry: whichever expands first fills it, and every later mention of the other definition reuses it. The implementation comment acknowledges the missing module qualifier but judges the collision "not yet triggerable" (`TType.sk:3048--3050`, upstream #889). That judgement is refuted by execution — registry row **E-EXPKEY**, violated and witnessed end-to-end in both failure directions (issue report `expansion-key-collision.md`):

- *Unsound acceptance.* Module `dep` exports `type Box = { s: string }` and `get(b: Box): string { return b.s; }`; the importing module declares its own `type Box = { n: number }`, binds `const b: Box = { n: 1 }` (filling the cache under ExpansionKey(`"Box"`, [])), then calls `get(b)`. The parameter's `Box` reuses the cached local body, the call is accepted with zero diagnostics, and at runtime `get` returns `b.s === undefined` at static type `string`. tsc rejects the same program (TS2741). Renaming the local alias — the only change — makes SKJS reject the call ("The field 's' is missing"), isolating the collision as the cause.
- *False rejection (dual).* Reordering so the imported `Box` expands first makes the *local* declaration `const b: Box = { n: 1 }` fail against `{ s: string }` — with the diagnostic anchored at the *other module's* definition, direct evidence that the cached entry crossed definitions.

Order-dependence plus wrong-module diagnostic positions pin the mechanism to the shared name-keyed cache rather than name resolution. Within a single module the premise holds — every reference to one spelling resolves to one definition (resolution takes the first matching definition, so even a stub/local name clash picks one winner consistently) — so every single-module result below is unaffected; the violation requires two modules checked under one substitution. The fix is to key by definition identity (module qualifier or defMap definition id), as #889 anticipates. Until then, every use of Lemma 15 and Theorem 19 carries **E-EXPKEY** as a premise.

One adjacent observation from the refutation search (name resolution, not the cache — outside this paper's scope, recorded so it is not lost): a module-local `type Function` does *not* shadow the stub-declared global `Function` — an annotation naming it resolves to the stub ("Missing call signature"), where tsc resolves to the module-local alias.

5.2 Symbolic arguments, precisely

Definition 14 (Symbolic in the Arguments)**.** instantiate$(n, \bar{a})$ *is* symbolic in the arguments *if there is a type-with-holes* $B_n[\cdot_1, \ldots, \cdot_k]$*, determined by the definition of* n *alone, such that for every* $\bar{a}$ *and every state* ρ*,* instantiate$(n, \bar{a})$ *run at* ρ *returns* $B_n[\bar{a}]$ *— the same* TType *value — where each hole occurrence is filled by the argument term* a_i *either (i) verbatim (up to position, which is not hashed), or (ii) as a variable occurrence of* a_i*'s variable with occurrence-level constraints added (`setConstraint` — a functional update, registry row SETCONSTRAINT-FUN), or (iii) replaced by a* binding-independent *approximation (a fixed type not depending on* ρ*). Diagnostics emitted along the way may depend on* ρ*; the returned value may not.*

Lemma 15 (Symbolic ⇒ Cache-Reuse Exactness)**.** *Assume* ***E-EXPKEY*** *for the reuse in question: the fill-site and use-site mentions resolve to the* same *definition of* n*. If* instantiate$(n, \bar{a})$ *is symbolic*

*in the arguments, then for any states ρ_0 (cache fill) and ρ' (use), the cached value equals what a fresh run at ρ' would return; hence $[\![cached]\!]_{\rho'} = [\![fresh]\!]_{\rho'}$ and the `TSInst` one-step containment under reuse follows from **E-NORM** at ρ' alone.*

Proof. Immediate: both runs return the value $B_n[\bar{a}]$ of the shared definition. The point of the symbolicity definition is that argument *bindings* are never consulted, so the variables inside $\bar{a}$ remain free in the cached term and are re-resolved through `get` at every later expansion step of the *user* of the cached term — never frozen into the cache. Without the **E-EXPKEY** premise the conclusion is simply false — the fresh run would expand a different $B_{n'}$ (Section 5.1). □

5.3 The alias path is binding-blind

The mechanism turns out to be simpler and stronger than the comment's operation-by-operation argument: on the `TypeDef` path, normalization *never consults the live substitution at all.*

Definition 16 (Binding-Blind)**.** *A computation is* binding-blind *if every `Subst` whose binding map it consults (via `get`, `peek`, `isUnbound`, `isOpen`, `isClosed`) is freshly created with the empty state, or is a clone whose results influence diagnostics only.*

Lemma 17 (normalize is Binding-Blind)**.** *`TEnv::normalize` and the helpers it reaches on the `TypeDef` path are binding-blind. By arms of the dispatch (`TEnv.sk:1755--2153`):*

- Literals, primitives, top, bottom, tuples-as-names, and the fatal arms*: these construct constants or `TType::inst`; no substitution is involved.*
- `TAInst` *(2011): resolves through the lexical `bindings` map — a `TEnv` field, not a substitution — returning the bound term via `withPos` (2030; verbatim insertion, case (i) of Definition 14); or emits an unexpanded `TType::inst` (2115); the reference-site constraint check (2103–2114) runs on `Subst::mcreate(tcenvb)` (2105)* — empty state — *and affects diagnostics only. The enum-member sub-arm (2045–2070) returns the member's literal type, computed by pure constant folding (Lemma 18).*
- `TAUnion`/`TAInter` *(2117–2132): both create a fresh empty `Subst` (2118, 2126). `union` never reads bindings at any level (Lemma 22). `inter` under the empty state sees every variable as unbound: bare-variable operands take the `setConstraint/unexpandedInter` arms (case (ii)); variables nested in compound operands are sealed to top* in the throwaway state *and thus replaced by the binding-independent ⊤ (case (iii)) — see Finding 23.*
- `TAKeyOf` *(1823–1826): creates a fresh empty `Subst` (1824) and expands the operand under it; see Finding 24. The indexed-access element read `T[number]` (2134–2141) has the same shape: fresh empty `Subst` (2139), covariant expansion in the throwaway state.*
- `TATemplate` *(1918): `normalizeTemplate` (2168) creates one fresh empty `Subst` (2175) and judges every hole by expanding it in mode* `none` *under that state (`templateHole`, 2277–2278): variables stay* residual *(no read at all — a clause with a nonempty var-set or a residual instance marks the hole irreducible, 2292–2297), and the reduction itself is pure string assembly over the literal sets. Case (iii) of Definition 14 holds with the approximation being the reduced literal union, a function of the hole* types *only.*
- `SKJS_Closed` *(2002–2010): `closeObjectType` (3155) creates a fresh empty `Subst` (3160) and expands the operand covariantly under it (3172); a variable nested in the operand seals*

to top in the throwaway state, and an abstract *operand is caught by the nonempty-var-set guard (3182–3190) and returned unclosed (diagnostics only outside definition validation). The closing itself rewrites index fields to ⊥ — no binding reads.*

- `TAObject`/`TAFun`/`TATypeOf`/tuples*: recursive normalization plus fresh empty substitutions at every operation site (`TEnv.sk:2459, 2472, 2487, 2548, 2681, 2840, 2920`); the mapped-type key analysis (`FNIn`, 2900) expands the key type in mode* none *under a fresh empty `Subst`, leaving variables residual and unread.*

impl: `TEnv.sk / normalize (1755), Subst creation sites (1824, 2105, 2118, 2126, 2139, 2175, 2459, 2472, 2487, 2548, 2681, 2840, 2920, 3160)`

Lemma 18 (The Enum Arm is Binding-Blind)**.** instantiate *on an `Enum` definition returns the union of the members' literal types, and an enum-member type reference returns one member's literal type. Member values are computed by `enumMembers` — constant folding over the declaration ASTs, resolving cross-enum references through `defMap` with a `visiting` cycle-breaker — which performs no substitution reads at all (there is no `Subst::get`, `peek`, or expansion call anywhere in `TypeCheckEnum.sk`); the heterogeneous-enum join goes through `TType::union`, binding-free by Lemma 22. So the arm is symbolic trivially: its value is a function of the declarations alone.*
impl: `TypeCheckEnum.sk / enumMembers (201), enumUnionType (487), enumMemberType (533)`

Theorem 19 (Alias-Path Expansion Invariance)**.** *For a `TypeDef` or `Enum` definition d resolved for n,* instantiate$(n, \bar{a})$ *is symbolic in the arguments (Definition 14); in fact it returns the same* TType *value at every state — a function of the* resolved definition d*, the argument terms $\bar{a}$, and the compilation unit. Consequently (Lemma 15) `seenExpansion` reuse is* exact *on the alias path* among mentions resolving to the same definition*; the key's failure to separate definitions is exactly **E-EXPKEY** (Section 5.1), not a failure of this theorem.*

Proof. instantiate on the `TypeDef` arm performs: parameter binding (`instantiateTParams`) — the returned bindings are the argument terms themselves, defaults normalized recursively, and the constraint check runs on `subst.clone()` of the state *copy* and is discarded (`TEnv.sk:378`, diagnostics only); then `normalize` of the body, which is binding-blind by Lemma 17. The `Enum` arm is binding-blind by Lemma 18. A binding-blind computation's result cannot depend on the ambient binding map; the remaining inputs (definition ASTs via `defMap`, the lexical bindings, the argument terms) are fixed by d, $\bar{a}$, and the compilation unit. The memo caches consulted along the way (`seenInter`/`seenUnion`) belong to the *fresh empty* substitutions of Lemma 17 and start empty at each site, so they cannot import state-dependence. □

Corollary 20 (Immunity to the Freeze-Window Hazard)**.** *Papers B/E of this series identify a hazard family in the `Subst` memo caches: under `freezeCandidates`, `Subst::get` returns an open candidate's bound* without sealing it*, so a cache entry whose value depends on that read (`seenSub`, `seenInter`) can go stale when the candidate later grows. On the alias path, `seenExpansion` is* immune by construction*: by Theorem 19 the cached value does not depend on any binding read, frozen or not — the unsealed read can return whatever it likes without being able to reach the cached value. This immunity is necessary because cross-mention keying makes cache hits routine. It is also orthogonal to **E-EXPKEY**: the collision serves a* wrong definition's *correct expansion, never a stale one — no binding staleness is involved in either direction. (The class path is exactly where the immunity argument breaks; Obligation 25.)*

Corollary 21 (Coherence Under save/restore/clone)**.** *`seenExpansion` is part of `SubstState`: `save`, `restore`, and `clone` carry bindings and cache atomically, so an entry filled on an exploratory*

path is rolled back with that path's bindings, and entries filled in a clone die with the clone. On the alias path this atomicity is unnecessary — by Theorem 19 even a leaked entry would be correct — but it is what keeps the class-path *exposure — and the* ***E-EXPKEY*** *collision exposure — bounded to reuse within one live substitution's lifetime.*

impl: `TTypeSubst.sk / SubstState::restore (40), SavedSubst::create (63), Subst::clone (149)`

5.4 Verifying the comment's three claims

Lemma 22 (Union Never Resolves a TVar)**.** *`TType::union` and everything it reaches perform no binding reads: `union_` unions the instance sets verbatim (`TInsts::union`) and merges clauses by var-set key; the recursive field/function cases (`TField::union`, `TFun::union`) recurse through `union` on covariant components — and through `inter` on contravariant ones, which is where the next lemma takes over. The `seenUnion` cache comment ("any TVar in the operands passes through unchanged") is confirmed at the `TType` level.*

impl: `TTypeUnion.sk / union (22), union_ (42), TInsts::union (167)`

Finding 23 (The comment's `inter` claim is imprecise)**.** *We attempted to verify "inter never resolves an unbound TVar" verbatim and* failed*: it is true only for* bare *variable operands (`inter_`'s guard arms, `TTypeInter.sk:138--153`, which attach the other operand as an occurrence constraint). A variable nested in a* compound *operand — e.g. (α | `undefined`) & `string` — falls through to `expandToClauses(Covariant())` on both operands (`TTypeInter.sk:158--159`), which* does *resolve α: sealing it to top if unbound, reading its value if bound. The claim is rescued where the cache needs it by Lemma 17: during alias normalization `inter` runs under a fresh* empty *state, so the nested variable is always unbound there, the seal lands in the throwaway state, and the substituted ⊤ is binding-independent — and replacing a conjunct by ⊤ over-approximates the intersection, so* ***E-NORM****'s direction is unharmed. Outside normalization (live `inter` during checking) the compound-operand resolution is real but is paper B's subject, not the cache's. Suggested action: reword the comment's parenthetical to "union caches independently of growth; inter attaches* bare *operand variables as constraints, and during normalization runs on an empty substitution, so nested variables are top-approximated, never bound-resolved."*

Finding 24 (The `keyof` claim holds vacuously — it never reads)**.** *The comment says `keyof` "approximates an abstract or unresolved operand identically (`string | number`) whether or not the variable is bound." The audit shows something stronger: the `TAKeyOf` arm creates a fresh* empty *`Subst` (`TEnv.sk:1824`) and `keyOfType` expands the operand under it, so a variable argument is* always *treated as unbound — its covariant read seals it to top in the throwaway state, the expansion has a top clause, and the guard at `TEnv.sk:3249--3258` returns `string | number`. The live binding is unreachable, so cached and fresh values agree not because the two reads approximate identically but because there is no read. (The result also covers rigid parameters, whose clauses have nonempty var-sets — the same guard.) Two caveats, both outside the cache theorem: `keyOfType`'s own over-approximation contract explicitly excludes validity checking ("do not use keyof to check that a key is valid," `TEnv.sk:3240--3246`), and symbol keys of an unresolved operand are not covered by `string | number` — both are* ***E-NORM****-side concerns about what the approximation means, not about cache invariance.*

impl: `TEnv.sk / normalize TAKeyOf arm (1823--1826), keyOfType (3247)`

5.5 Where the proof stops: the class path

Obligation 25 (E-CLASS-SYM: Class Instantiation Symbolicity)**.** *On the `Class` arm,* instantiate *calls `new` → `newObject` → `addFields` passing the* copy of the live state *made at `TEnv.sk:196` — not*

a fresh empty substitution. Field-type normalization inside is binding-blind (Lemma 17 applies arm by arm), but member assembly is not obviously so: any `TType::inter` *reached with that substitution — the audited candidates are index merging (*`TField::interIndex`*,* `TTypeInter.sk:245`*, called unconditionally on* `TType::inter`*, unlike* `TField::inter`*, which defers via* `unexpandedInter` *when* `tenv.expandInter` *is false), constructor assembly (*`findConstructor`*), and* `TIndexes::create` *— can reach* `expandToClauses(Covariant())` *on a compound operand mentioning a type argument's variable and read its* current *binding. Because the read happens in the discarded copy, the seal that would normally freeze the variable and justify the read (paper's never-widen lemma (iii)) is thrown away: the live variable can still grow, and the cached class body would then diverge from a fresh recomputation — the same staleness family as the freeze-window hazard (Corollary 20), reachable here even without* `freezeCandidates`*, and additionally through it.* ***Status: open.*** *We attempted both directions and completed neither: the proof attempt dies at the* `interIndex` *chain above, and a concrete end-to-end counterexample (a class instantiated at a bound-open candidate whose member merge reads the bound, cached, followed by observable growth) was not constructed. Discharge either by (a) showing the reachable* `inter` *operands during* `addFields` *never nest a live inference variable inside a compound type (targ variables occur bare or inside unexpanded* `TSInst`*s there), or (b) making the class path binding-blind by construction (fresh empty substitution in* `newObject`*, as the alias path already effectively has), or (c) keying* `seenExpansion` *entries for class instances by substitution version, as* `seenSub` *already is for cycles.*

impl: `TEnv.sk / instantiate (196, 224), newObject (901), newObjectOf (919), newFieldsOf (952), addFields (1246), findConstructor (1585)`

impl: `TTypeInter.sk / TField::interIndex (245), inter_ (158)`

Obligation 26 (E-GUARD: Identity Returns Must Not Poison the Live Cache)**.** instantiate *returns the unexpanded* TSInst *itself in two cases: the recursion guard, when* `seenInstantiate` *already contains n (*`TEnv.sk:189--191`*), and the circular-alias guard (*`isCircularTypeDef`*,* `TEnv.sk:212--214`*);* `expandToVectors` *caches whatever* instantiate *returns (*`TType.sk:700`*). If such an* identity *entry is consulted from a long-lived substitution's cache, every later expansion of that instance hits the identity, whose sole instance is then dropped by the seen check — the named type contributes* nothing *(effectively ⊥) to that expansion. For the* recursion-guard *return this is empirically safe:* `seenInstantiate` *is nonempty only inside an* instantiate *frame using the discarded substitutions of Lemma 17, and a recursive alias's inner* `TSInst` *sits nested in a field/argument the head worklist never processes (*`type A = A | number` *reads as* `number`*). For the* circular-alias *return the empty contribution is the intended denotation — an alias that requires itself denotes nothing, and the declaration is independently reported fatal at its own site (TS2456) — so only programs already rejected can observe it.* ***Status: open (small)*** *— the safety argument is a call-site audit, not an invariant. The recommended cache-write invariant is "the cached value's insts ≠ the key" (asserting* `seenInstantiate` *empty at the write does not cover the circular-alias return, which fires with it empty).*

impl: `TEnv.sk / instantiate (186--191, 212--214), isCircularTypeDef (495); TType.sk / expandToVectors (693--703)`

5.6 The missing-annotation placeholder atom

An un-annotated field of an *ambient* class has no type to normalize; the checker creates a synthetic name for it (`Missing_type_annotation_for_...`) and instantiate, on seeing that prefix, returns not an identity TSInst but an *opaque atom*: a synthetic rigid `TParam`, "a subtype of nothing but itself and `unknown`, with nothing but itself a subtype of it" (`TEnv.sk:183--185`;

`TType::missingAnnotationPlaceholder`, `TType.sk:212--217`). The atom's identity derives from the placeholder's name and declaration file — deliberately *not* from the class's type arguments, so the placeholder field of two instantiations of one generic ambient class is one atom (structurally identical at runtime, mutually assignable in tsc; comment at `TType.sk:206--211`).

The flows the atom closes are the assignability ones, and those are verified at the pin: reading such a field into any concrete annotation is rejected on the subtype side (the atom is below nothing but itself), and writing anything but the field's own value into it is rejected on the supertype side — in both directions the diagnostic surfaces the synthetic name. What remains open is the atom's behavior at the *membership-consulting* sites: the runtime value of the field is arbitrary, so soundness requires every site that asks "is v in $[\![\text{atom}]\!]$?" to treat the answer as unknown — the clause-with-nonempty-var-set shape the atom expands to must be handled as an unanalyzable remainder by truthiness, equality narrowing, and `typeof` refinement, not as empty. That site-by-site audit (refinement's residue and verdict paths on abstract clauses) has not been done; it is registry row **E-PLACEHOLDER-ATOM** (OPEN). No unsound path is known.

impl: `TEnv.sk / instantiate (174--185); TType.sk / missingAnnotationPlaceholder (212), kMissingTypeAnnotationPrefix (2093)`

6 Alias Cycles and Termination

Three guards keep expansion of recursive definitions finite; soundness of each cutoff is the point here, termination secondary.

One-level unfolding (`seenInstantiate`). While expanding n, an inner reference to n normalizes to an unexpanded `TSInst` instead of unfolding (`TEnv.sk:79--82, 189--192, 2089--2091`). The cutoff is sound by construction: the re-emitted instance denotes the named type ((N1)), and nested occurrences sit inside object fields or type arguments, where the worklist does not descend — expansion is one level deep by design (the main paper's "expansion rewrites a type to its instance-plus-clauses view" is inherently head expansion).

Self-requiring aliases (`isCircularTypeDef`). An alias that requires itself to compute — directly or through a partner (`type P<T> = Q<T>; type Q<T> = P<Q<T>>`) — can rewrite to a strictly larger instantiation on every unfolding, which no per-frame seen-set catches. instantiate detects the shape up front and hands back the unexpanded instantiation (`TEnv.sk:198--214`); the declaration itself is reported fatal (TS2456) at its own site, so the empty contribution is observable only in programs already rejected. Two neighbouring guards in parameter filling are fatal or top-recovering, hence trivially sound: a cycle through a second alias's *default* is refused as excessively deep (TS2589, `TEnv.sk:283--299`), a directly circular default resolves to `unknown`, so it cannot introduce a value at an unsupported type (`TEnv.sk:330--336`), and a self-referential *constraint* refuses the instantiation outright (TS2589, `TEnv.sk:360--368`).

The worklist *seen* set. A *top-level* recursive occurrence — `type A = A | number` normalizes to $(\{\mathsf{TSInst}(A)\}, \{\texttt{number}\})$ — is deduplicated: the body's $\mathsf{TSInst}(A)$ equals the instance being expanded and is dropped (`TType.sk:718--722`). The expansion of A is then just `number`. This is sound under a least-fixpoint reading of type definitions: the equation $X = X \cup [\![\texttt{number}]\!]$ has least solution $[\![\texttt{number}]\!]$, and any value's membership in a recursively defined type must be justified by finite unfolding. We record this reading as part of (N1)'s meaning rather than as a separate assumption; a definition intended coinductively (values justified only by infinite regress) has no

inhabitants a program can construct. Mutual cycles behave the same: expanding `type A = B; type B = A` yields empty clauses (bottom) after both identities deduplicate — the least fixpoint — while the checker independently rejects such definitions at validation (TS2456).

Fatal circularity (`forbiddenInsts`). Within one instantiate frame, a *second* reference to the in-expansion name is fatal (TS2456, `TEnv.sk:2032--2036`): the first reference marks the name forbidden (2089–2091), the next lookup dies. Generative loops (`type A<T> = A<A<T>> | number`) die the same way — normalizing the inner argument `A<T>` is a second reference. A fatal diagnostic ends checking, which is trivially sound. Termination of the worklist itself: each iteration either pops a *seen* instance's expansion products (each distinct instance is processed at most once) or returns $\top$; non-termination would require instantiate to keep manufacturing distinct instances at the top level of bodies, which the generative-loop fatal and the `isCircularTypeDef` guard cut off on the alias path. We have not proved a global termination bound for the class path (chains of distinct `TSInst`s through superclass clauses); termination is outside the soundness contract and is left unstated.

impl: `TEnv.sk / seenInstantiate (82), forbiddenInsts (106), normalize TAInst (2032, 2089); TType.sk / expandToVectors (718--722)`

7 Status and Ledger Edits

7.1 Decomposition of L-EXP

Item	Statement	Anchor
E-PLUMB	Worklist, clause rebuild, `mapSTypes` factoring, top short-circuit, residual pass-through (Lemmas 6, 7)	`expandToVectors`, `mapSTypes`
E-TVAR	Variable one-step containment, modes none/cov (Lemma 8); mode contra excluded with counterexample (Finding 9); refinement clients verified none/cov-only (Lemma 10)	`Subst::get`
E-INTER	`TInter` one-step containment; cycle residual exact (§4.3)	`TInter::expand`
E-NORM	Instantiation is semantics-preserving at the state of use (Axiom 11) — the `TEnv`/normalize interface	`TEnv::instantiate`
E-ARR	Array-literal commitment discipline (Axiom 12)	`TArrayLitTyping.sk`
E-SYM-ALIAS	Cache invariance, `TypeDef`/`Enum` paths: binding-blind, hence exact reuse among same-definition mentions, and freeze-window immunity (Theorem 19, Corollary 20)	`seenExpansion`; `TEnv::normalize`
E-EXPKEY	The key identifies one definition — refuted across modules, witnessed both ways (Section 5.1)	`ExpansionKey`; `expansionCacheKey`
E-CLASS-SYM	Cache invariance, `Class` path (Obligation 25)	`TEnv::newObject`; `TField::interIndex`
E-GUARD	Identity returns (recursion guard; circular alias) never consulted from a long-lived substitution (Obligation 26)	`expandToVectors` (700)

Item	Statement	Anchor
E-PLACEHOLDER-ATOM	The missing-annotation atom is treated as unknown at every membership-consulting site (Section 5.6)	`missingAnnotationPlaceholder`

Net effect on **L-EXP**: ASSUMED → PROVED for modes `none/cov`, conditional on **L-UNION**/**L-INTER**/**L-INTER-BOT**, **L-MONO**, **E-NORM**, **E-ARR**, **E-CLASS-SYM**, **E-GUARD**, **E-PLACEHOLDER-ATOM**, and — for any reuse crossing modules — **E-EXPKEY**, which is refuted, so cross-module programs sit outside the proved region until the key carries definition identity. The single-module residue is smaller and sharper than the original axiom: two named interface assumptions and three localized open obligations.

7.2 Registry

The authoritative, current statuses are maintained in the registry (`docs/ledger.yaml`), rendered as the main paper's Assumptions and Proof Obligations tables; this paper is cited there as evidence — in particular for the mode scoping of Axiom **L-EXP** (the containment holds for modes `None/Covariant` only, Lemma 10) and for the rows E-NORM, E-ARR, E-CLASS-SYM, E-GUARD, E-EXPKEY, E-PLACEHOLDER-ATOM. The registry rows this paper supports:

Row	Statement	Status
L-EXP	Expansion preserves or enlarges semantics (modes `None`/Covariant); residual instances keep theirs.	PROVED
E-NORM	Named-type normalization is semantics-preserving (interface consumed by expansion).	ASSUMED
E-ARR	Delayed array-literal forms keep their semantics under expansion.	ASSUMED
E-CLASS-SYM	Class expansion either does not depend on existing variable bindings, or any bindings it reads cannot make cache reuse unsound.	OPEN
E-GUARD	Recursion-guard identity expansions are never cached into long-lived substitutions.	OPEN
E-PLACEHOLDER-ATOM	An un-annotated ambient-class field reads as an opaque per-field atom; the atom's denotation must over-approximate the arbitrary runtime value.	OPEN
E-EXPKEY	The expansion-cache key (`ExpansionKey`: unqualified name text + arguments) identifies one definition: same-named, same-argument aliases from different modules must not collide.	VIOLATED

Imports (review contract). The named statements this paper assumes from elsewhere. A reviewer treats each as an axiom, checking citations against the registry's canonical statement; no other paper's internals are in scope:

Row	Canonical statement	Owner	Status
L-UNION	`union` over-approximates set union (all levels).	B	PROVED
L-INTER	When `inter` returns a non-$\bot$ result, that result contains the semantic intersection of its operands, provided every recursive call that returned $\bot$ had disjoint operands (L-INTER-BOT).	B	OPEN
L-INTER-BOT	A $\bot$ result from `inter` implies semantic disjointness.	B	VIOLATED
L-MONO	Legal inference-state changes during refinement preserve membership in types computed earlier.	E	PROVED
TVAR-SEM	A type-variable occurrence denotes its substitution envelope intersected with its occurrence constraints; an unbound envelope is empty.	main	PROVED
SETCONSTRAINT-FUN	`TVar::setConstraint` is a functional update on a frozen value: constraints are occurrence-level, not shared state.	E	PROVED
TENV-EXPANDINTER	The `tenv.expandInter` mode flag is path-scoped, and both intersection forms it selects (expanded and deferred `unexpandedInter`) satisfy the two-case L-INTER/L-INTER-BOT containment.	E	OPEN

Outstanding recommendations. (i) Repair **E-EXPKEY**: key `seenExpansion` by definition identity (module qualifier or defMap definition id), and correct the "not yet triggerable" judgement in the `ExpansionKey` comment (`TType.sk:3048--3050`) — the collision is witnessed. (ii) Fix the code comment `TTypeSubst.sk:17--31` per Finding 23 (inter's "never resolve" holds for bare operands; nested variables are top-approximated because normalization substitutions are empty) and Finding 24 (keyof never reads the live binding). (iii) Wire the E-GUARD pinned test with the "cached insts $\neq$ key" invariant; record Lemma 5's inventory (expansion performs get-transitions and cache writes only; never a grow) as the concrete content behind L-MONO's "evolutions of the kind covered". (iv) Run the E-PLACEHOLDER-ATOM site audit (refinement's residue and verdict paths on abstract clauses).

7.3 Findings from failed proof attempts (index)

Finding 9: unconditional (all-modes) L-EXP is false; counterexample unbound-variable contravariant read returning $\bot$. Section 5.1: the key-identity premise E-EXPKEY was refuted outright — the attempted proof ("name text identifies a definition") fails across modules, and the refutation ran end-to-end in both failure directions. Finding 23: the comment's blanket "inter never resolves an unbound TVar" failed verification; rescued only via the binding-blind property. Finding 24: the keyof-equal-approximation claim could not be proved as stated (a bound variable read *would* produce precise keys) — it is true for the vacuous reason that no live read occurs. Obligation 25: class-path symbolicity resisted both proof (dies at the `interIndex` chain) and refutation (no end-to-end counterexample constructed). Obligation 26: cache poisoning by identity returns could not be excluded by a local invariant; global call-site audit only. Section 5.6: the placeholder atom's membership-consulting sites are unaudited (E-PLACEHOLDER-ATOM).

Implementation Anchors (Summary)

Model	Implementation (rev. `bafdcf7c`)
expand$_m$, worklist	`TType.sk / expandToVectors (680)`
rebuilds	`expand (727), expandToClauses (739), mapSTypes (434), rawMap (408)`
get transitions, freeze	`TTypeSubst.sk / Subst::get (207), freezeCandidates (89)`
cache + saved state	`TTypeSubst.sk / seenExpansion (31), restore (40), clone (149)`
cache key	`TType.sk / ExpansionKey (3055), expansionCacheKey (3057)`
cache read/write sites	`TType.sk / expandToVectors (693--703);` `TTypeSubtyping.sk / collectTVars (1529--1539)`
instantiate, parameter binding	`TEnv.sk / instantiate (168), instantiateTParams (257)`
placeholder atom	`TEnv.sk / 174--185; TType.sk / missingAnnotationPlaceholder (212)`
symbolic resolution	`TEnv.sk / normalize TAInst arm (2011--2116), bindings (46)`
fresh empty substitutions	`TEnv.sk / 1824, 2105, 2118, 2126, 2139, 2175,` `2459, 2472, 2487, 2548, 2681, 2840, 2920, 3160`
keyof	`TEnv.sk / keyOfType (3247)`
template literals	`TEnv.sk / normalizeTemplate (2168), templateHole (2277)`
`Closed<T>`	`TEnv.sk / closeObjectType (3155)`
enums	`TypeCheckEnum.sk / enumMembers (201), enumUnionType (487)`
mapped types	`TEnv.sk / FNIn arm (2900)`
class path	`TEnv.sk / new (842), newObject (901), addFields (1246)`
intersections	`TTypeInter.sk / TInter (17), expand (37), inter_ (129), interIndex (245)`
unions	`TTypeUnion.sk / union (22), union_ (42)`
array literals	`TArrayLitTyping.sk / expand (100)`
recursion + circularity guards	`TEnv.sk / seenInstantiate (82), forbiddenInsts (106), isCircularTypeDef (495)`